Name : Manaswi Kabadi
Class : D15C
Roll no. : 27
Batch : B

EXP - 10

Aim: Explore and implement Autoencoders for image denoising.

Dataset Description:

The Fashion MNIST Dataset used in this experiment is a built-in dataset provided by the Keras deep learning library through TensorFlow. It is a widely used benchmark dataset for image classification tasks in Computer Vision. The dataset consists of grayscale images of clothing items belonging to 10 different fashion categories. Each image is of size 28×28 pixels and represents a single product from Zalando's fashion catalog. Although the original dataset is designed for supervised learning (each image has a label from 0 to 9), in this experiment the labels are not used because the task involves training a Denoising Autoencoder, which is an unsupervised learning approach. The model learns to reconstruct clean images from noisy inputs, meaning the training process uses the image itself as the target output.

- File Type: Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- Dataset Size: 70,000 grayscale images (60,000 training samples + 10,000 testing samples)
- Data Format: 28×28 grayscale pixel images (integer values 0–255, normalized to 0–1 in this experiment)
- Number of Classes: 10 Fashion Categories (T-shirt/top, Trouser, Pullover, Dress, etc.)

Column Name	Data Type	Description
Image	Numerical (28×28 matrix)	Grayscale pixel values representing a fashion item image (normalized between 0 and 1 in preprocessing).
Label	Numerical (Integer: 0–9)	Category index representing the type of clothing item (not used in this autoencoder experiment).

Dataset Source: https://keras.io/api/datasets/fashion_mnist/

Autoencoders

Theory:

Autoencoders are a class of unsupervised deep learning models designed to learn efficient representations of input data by reconstructing the input at the output layer. They consist of two main components: an encoder and a decoder. The encoder compresses the input into a lower-dimensional

latent representation, while the decoder reconstructs the original input from this representation. Let the input image be represented as:

$$x \in \mathbb{R}^{(H \times W \times C)}$$

where H = height of the image, W = width of the image, C = number of channels.

In this experiment, $x \in \mathbb{R}^{(28 \times 28 \times 1)}$

In a Denoising Autoencoder, instead of feeding the clean image directly, we first corrupt it with noise:

$$\tilde{x} = x + \varepsilon$$

The model is then trained to reconstruct the clean image x from the noisy input.

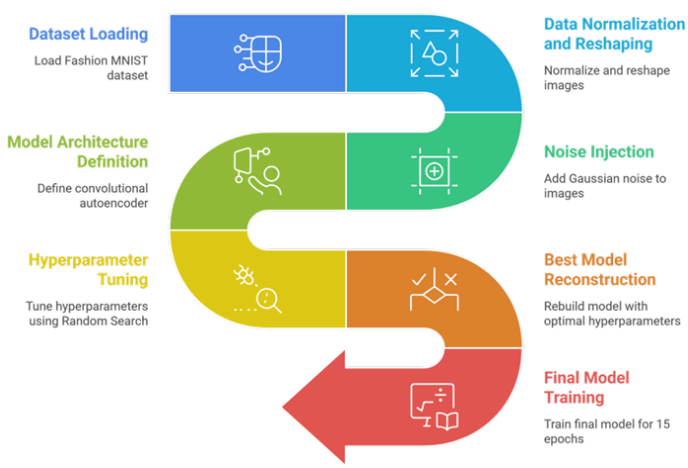
Limitations:

1. **Information Loss Due to Bottleneck Compression:** Autoencoders compress the input into a lower-dimensional latent representation z. If the latent space dimension is too small, important structural details may be lost during encoding. This can result in blurry or incomplete reconstructions, especially for complex images. In denoising tasks, excessive compression may remove fine details along with noise.
2. **Overfitting to Training Data:** Autoencoders aim to minimize reconstruction loss. If the model is too complex (large number of filters or layers), it may simply memorize the training images instead of learning meaningful feature representations. This reduces generalization performance when tested on unseen noisy images.
3. **Blurry Reconstructions with Pixel-wise Loss Functions:** When using losses such as Mean Squared Error (MSE) or Binary Cross-Entropy (BCE), the model optimizes pixel-level similarity. This often produces overly smooth or blurry outputs because the network averages possible reconstructions rather than preserving sharp edges and textures.
4. **Limited Ability to Capture Complex Data Distributions:** Standard autoencoders do not explicitly model the probability distribution of the data. Unlike Variational Autoencoders (VAEs), they do not enforce a structured latent space. As a result, learned representations may lack smoothness or interpretability, limiting their generative capabilities.

Workflow:

5. **Dataset Loading:** The built-in Keras Fashion MNIST dataset is loaded using TensorFlow. It contains 60,000 training images and 10,000 testing images of grayscale fashion items (28×28 pixels). Since this experiment focuses on denoising autoencoders, the labels are ignored and only image data is used.
6. **Data Normalization and Reshaping:** Pixel values are converted from integers (0–255) to floating-point values in the range [0, 1] by dividing by 255. Each image is reshaped to include a channel dimension, converting the shape from (28, 28) to (28, 28, 1). This ensures compatibility with convolutional layers, which expect 3D image tensors.
7. **Noise Injection (Denoising Setup):** Random Gaussian noise is added to the training and testing images: $\tilde{x} = x + \varepsilon$. The noisy images are clipped to remain within the valid pixel range [0, 1]. The noisy images act as inputs, while the original clean images serve as target outputs for training.

8. **Model Architecture Definition (Convolutional Autoencoder):** A function build_model(hp) defines a convolutional autoencoder with tunable hyperparameters. The model is compiled using:
    - Optimizer: Adam (learning rate tuned: 1e-2, 1e-3, 1e-4)
    - Loss Function: Binary Cross-Entropy
9. **Hyperparameter Tuning (Keras Tuner – Random Search):** The RandomSearch tuner explores combinations of:
    - Number of convolution filters
    - Learning rate
    - Maximum trials: 5
    - Objective: Minimize validation loss
    - Early stopping is used (patience = 3) to prevent unnecessary training if validation loss does not improve.
10. **Initial Search Phase:** Each hyperparameter configuration is trained for 10 epochs with a validation split of 20%. The tuner evaluates performance and identifies the best hyperparameter combination based on validation loss.
11. **Best Model Reconstruction:** The optimal hyperparameters are retrieved using tuner.get_best_hyperparameters(). The autoencoder is rebuilt using these best-performing settings.
12. **Final Model Training:** The best model is trained for 15 epochs using:
    - Batch size = 128
    - Shuffling enabled
    - Validation performed on the full noisy test dataset



Code:

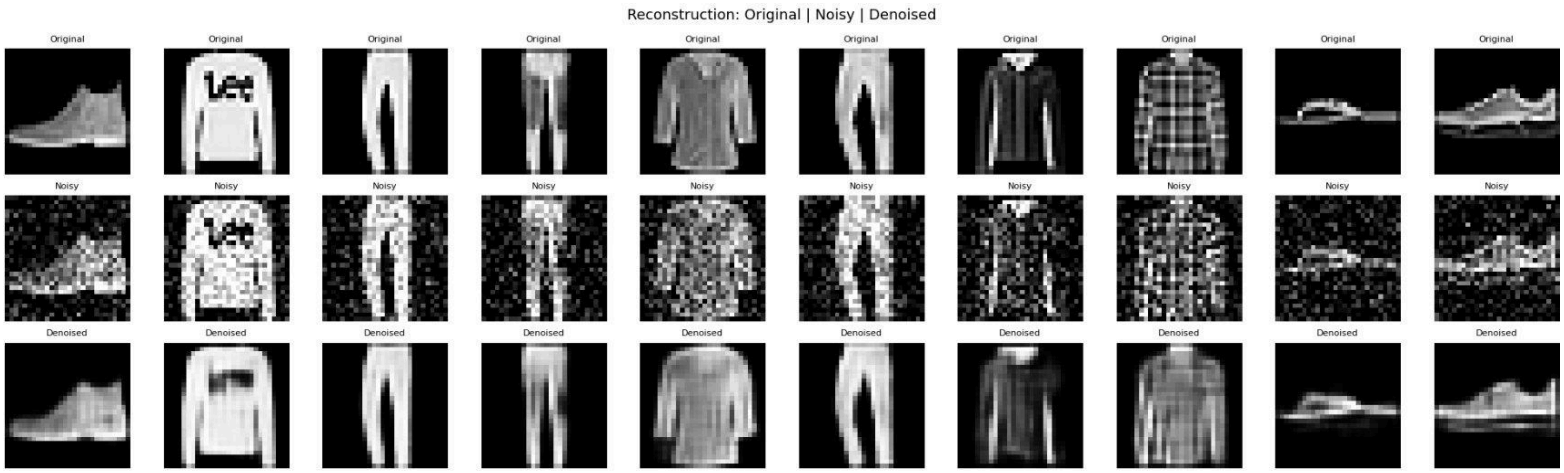
```
!pip install keras-tuner --quiet
import tensorflow as tf
from tensorflow.keras import layers, models
import keras_tuner as kt
import numpy as np
import matplotlib.pyplot as plt
```

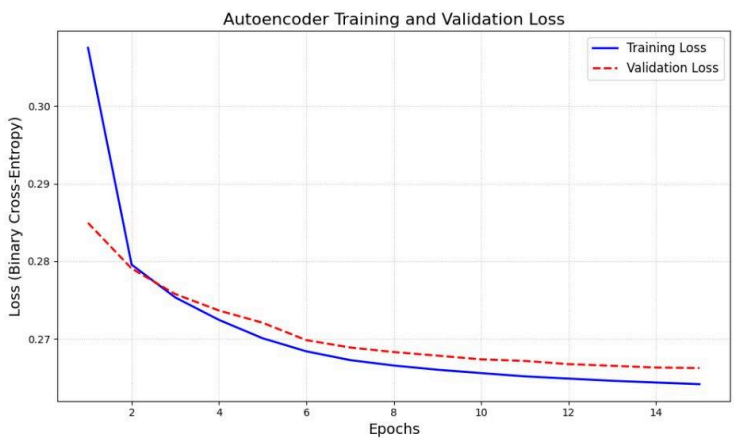
```python
from tensorflow.keras.utils import
plot_model

from IPython.display import Image

(x_train, _), (x_test, _) =
tf.keras.datasets.fashion_mnist.load_da
ta()

x_train = x_train.astype('float32') /
255.

x_test  = x_test.astype('float32') /
255.

x_train = np.expand_dims(x_train,
axis=-1)   # (60000, 28, 28, 1)

x_test  = np.expand_dims(x_test,
axis=-1)   # (10000, 28, 28, 1)

print("Training data shape :",
x_train.shape)

print("Testing  data shape :",
x_test.shape)

noise_factor = 0.2

x_train_noisy = x_train + noise_factor
* tf.random.normal(shape=x_train.shape)

x_test_noisy  = x_test  + noise_factor
* tf.random.normal(shape=x_test.shape)

x_train_noisy =
tf.clip_by_value(x_train_noisy,
clip_value_min=0., clip_value_max=1.)

x_test_noisy  =
tf.clip_by_value(x_test_noisy,
clip_value_min=0., clip_value_max=1.)

plt.figure(figsize=(20, 4))

for i in range(10):

    ax = plt.subplot(2, 10, i + 1)

    plt.imshow(tf.squeeze(x_train[i]),
cmap='gray')

    plt.title("Original", fontsize=8)

    plt.axis("off")


    ax = plt.subplot(2, 10, i + 11)

plt.imshow(tf.squeeze(x_train_noisy[i])
, cmap='gray')

    plt.title("Noisy", fontsize=8)

    plt.axis("off")

plt.suptitle("Sample Images: Original
vs Noisy", fontsize=13)

plt.tight_layout()

plt.show()


def build_model(hp):

    inputs = layers.Input(shape=(28,
28, 1))

    hp_filters = hp.Int('filters',
min_value=16, max_value=64, step=16)

    x = layers.Conv2D(hp_filters, (3,
3), activation='relu',
padding='same')(inputs)

    x = layers.MaxPooling2D((2, 2),
padding='same')(x)

    x = layers.Conv2D(hp_filters // 2,
(3, 3), activation='relu',
padding='same')(x)

    encoded = layers.MaxPooling2D((2,
2), padding='same')(x)

    x = layers.Conv2D(hp_filters // 2,
(3, 3), activation='relu',
padding='same')(encoded)

    x = layers.UpSampling2D((2, 2))(x)

    x = layers.Conv2D(hp_filters, (3,
3), activation='relu',
padding='same')(x)

    x = layers.UpSampling2D((2, 2))(x)

    decoded = layers.Conv2D(1, (3, 3),
activation='sigmoid',
padding='same')(x)

    model = models.Model(inputs,
decoded)
```

```
    hp_learning_rate =
hp.Choice('learning_rate',
values=[1e-2, 1e-3, 1e-4])

    model.compile(


optimizer=tf.keras.optimizers.Adam(lear
ning_rate=hp_learning_rate),

        loss='binary_crossentropy')

    return model

tuner = kt.RandomSearch(

    build_model,

    objective='val_loss',

    max_trials=5,

    executions_per_trial=1,

    directory='autoencoder_tuning',
project_name='denoising_fashion_mnist')

stop_early =
tf.keras.callbacks.EarlyStopping(monito
r='val_loss', patience=3)

print("\n===== Starting Hyperparameter
Search =====\n")

tuner.search(

    x_train_noisy, x_train,

    epochs=10,

    validation_split=0.2,

    callbacks=[stop_early],

    verbose=1)

best_hps =
tuner.get_best_hyperparameters(num_tria
ls=1)[0]

print(f"\nBest number of filters :
{best_hps.get('filters')}")

print(f"Best learning rate     :
{best_hps.get('learning_rate')}")

best_model =
tuner.hypermodel.build(best_hps)

best_model.summary()

print("\n===== Final Model Training
=====\n")

history = best_model.fit(

    x_train_noisy, x_train,

    epochs=15,

    batch_size=128,

    shuffle=True,

    validation_data=(x_test_noisy,
x_test),

    verbose=1)

decoded_imgs =
best_model.predict(x_test_noisy)

n = 10

plt.figure(figsize=(20, 6))

for i in range(n):

    # Original

    ax = plt.subplot(3, n, i + 1)

    plt.imshow(tf.squeeze(x_test[i]),
cmap='gray')

    plt.title("Original", fontsize=8)

    plt.axis("off")

    # Noisy

    ax = plt.subplot(3, n, i + 1 + n)
plt.imshow(tf.squeeze(x_test_noisy[i]),
cmap='gray')

    plt.title("Noisy", fontsize=8)

    plt.axis("off")

    # Denoised

    ax = plt.subplot(3, n, i + 1 + 2*n)
plt.imshow(tf.squeeze(decoded_imgs[i]),
cmap='gray')

    plt.title("Denoised", fontsize=8)

    plt.axis("off")

plt.suptitle("Reconstruction: Original
| Noisy | Denoised", fontsize=13)

plt.tight_layout()
```

```
plt.show()

train_loss = history.history['loss']

val_loss   =
history.history['val_loss']

epochs_range = range(1, len(train_loss)
+ 1)

plt.figure(figsize=(10, 6))

plt.plot(epochs_range, train_loss,
'b-', label='Training Loss',
linewidth=2)

plt.plot(epochs_range, val_loss,
'r--', label='Validation Loss',
linewidth=2)

plt.title('Autoencoder Training and
Validation Loss', fontsize=16)

plt.xlabel('Epochs', fontsize=14)

plt.ylabel('Loss (Binary
Cross-Entropy)', fontsize=14)

plt.legend(loc='upper right',
fontsize=12)

plt.grid(True, linestyle=':',
alpha=0.7)

plt.tight_layout()

plt.show()

plot_model(

    best_model,
to_file='autoencoder_architecture.png',

    show_shapes=True,

    show_layer_names=True,

    show_layer_activations=True,

    dpi=100)

Image('autoencoder_architecture.png')

index = 0

isolated_noise = x_test_noisy[index] -
decoded_imgs[index]

plt.figure(figsize=(15, 5))

ax = plt.subplot(1, 3, 1)

plt.imshow(tf.squeeze(x_test_noisy[inde
x]), cmap='gray')

plt.title("Noisy Input")

plt.axis("off")

ax = plt.subplot(1, 3, 2)

plt.imshow(tf.squeeze(decoded_imgs[inde
x]), cmap='gray')

plt.title("Denoised Output")

plt.axis("off")

ax = plt.subplot(1, 3, 3)

plt.imshow(tf.squeeze(isolated_noise),
cmap='gray')

plt.title("Isolated Noise (Removed)")

plt.axis("off")

plt.suptitle("Noise Isolation
Analysis", fontsize=13)

plt.tight_layout()

plt.show()

print("\n========== Training Summary
==========")

for ep in range(len(train_loss)):

    print(f"Epoch {ep+1:2d} | Train
Loss: {train_loss[ep]:.4f} | Val Loss:
{val_loss[ep]:.4f}")

print(f"\nBest Filters      :
{best_hps.get('filters')}")

print(f"Best Learning Rate:
{best_hps.get('learning_rate')}")

print(f"Final Train Loss  :
{train_loss[-1]:.4f}")

print(f"Final Val Loss    :
{val_loss[-1]:.4f}")

print(f"Generalization Gap:
{abs(val_loss[-1] -
train_loss[-1]):.4f}")
```

Output:





Performance Analysis:

The performance of the hyperparameter-tuned Convolutional Denoising Autoencoder is evaluated using validation loss during tuning, training vs validation loss curves, and qualitative reconstruction results. Since this is an unsupervised image reconstruction task, evaluation focuses on reconstruction loss (Binary Cross-Entropy) and visual denoising quality rather than classification accuracy.

1. Hyperparameter Tuning Performance:

- Best Validation Loss (during tuning): 0.2629
- Best Number of Filters: 64
- Best Learning Rate: 0.001

The tuner explored 5 different configurations and selected:

- A relatively higher filter size (64) → allowing richer feature extraction.
- A moderate learning rate (0.001) → providing stable and efficient convergence.

This indicates that higher representational capacity improved reconstruction performance without causing instability.

2. Final Training Performance (15 Epochs):

Final Epoch Results (Epoch 15):

- Training Loss: 0.2621

- Validation Loss: 0.2644

Loss Trend Observation:

- Epoch 1 — Training Loss: 0.3536, Validation Loss: 0.2818. Rapid initial learning.
- From Epoch 2 onward: Steady and smooth decline in both training and validation loss. No sudden spikes or divergence. Loss gradually stabilizes around ~0.264.

This shows stable convergence, no training instability, and good optimizer configuration.

3. Generalization Analysis:

- Training Loss (Final): 0.2621
- Validation Loss (Final): 0.2644
- Difference ≈ 0.0023

This very small gap indicates minimal overfitting, strong generalization capability, and that the model is not memorizing training images. Noise removal patterns learned are transferable to unseen test images. Unlike classification tasks where large training–validation gaps signal overfitting, here the near-overlapping curves indicate balanced model capacity.

Hyperparameter Tuning:

Convolutional Denoising Autoencoders are sensitive to architectural and optimization hyperparameters because they directly influence feature extraction capacity, convergence stability, reconstruction quality, and generalization ability. Manual tuning is inefficient due to the combination space of filter sizes and learning rates. Therefore, automated hyperparameter tuning is performed using Keras Tuner with the Random Search algorithm. The tuner evaluates multiple autoencoder configurations and selects the best-performing model based on validation loss (Binary Cross-Entropy). The tuned hyperparameters in this experiment are:

- **Number of Convolutional Filters:** The number of filters in the first convolutional layer is tuned between: 16, 32, 48, 64. Filters determine the model's ability to extract spatial features such as edges, textures, and object structures. The optimal configuration selected: Best Number of Filters: 64. This indicates that richer feature maps improved denoising performance, enabling the encoder to learn more expressive latent representations.
- **Learning Rate:** The learning rate of the Adam optimizer is tuned among: 0.01, 0.001, 0.0001. The learning rate controls the step size during gradient descent. The tuner selected: Best Learning Rate: 0.001. This value provided stable convergence with smooth loss reduction and no oscillations

Conclusion:

This experiment successfully implemented a Convolutional Denoising Autoencoder to remove Gaussian noise from Fashion MNIST images. The hyperparameter-tuned model (64 filters, learning rate = 0.001) achieved a low validation loss (~0.2644) with minimal gap between training and validation loss, indicating strong generalization and stable convergence. The reconstructed images clearly demonstrate effective noise removal while preserving essential structural features. Overall, the results confirm that autoencoders are powerful unsupervised models for image denoising tasks.